Rapport

Avant-propos

Lancement de l'application

Afin d'exécuter l'application, il est nécessaire d'utiliser les commandes Docker suivantes.

- **sudo docker compose down -v**
  Cette commande permet d'arrêter et de supprimer les conteneurs. L'option -v permet également de supprimer les volumes associés. Sans cette option, les données restent persistantes.
- **sudo docker compose up --build -d**
  Cette commande permet de construire les images Docker si nécessaire puis de démarrer l'ensemble des conteneurs définis dans le fichier docker-compose.yml.

Ces commandes permettent ainsi de redémarrer l'application dans un environnement propre ou de relancer l'ensemble des services nécessaires à son fonctionnement.
Note : sudo docker compose build suivi de sudo docker compose up -d = 2e commande

I - Présentation des fonctionnalités

A) Section connexion/inscription

Il en existe deux possibles: une pour l'inscription et une autre pour la connexion.

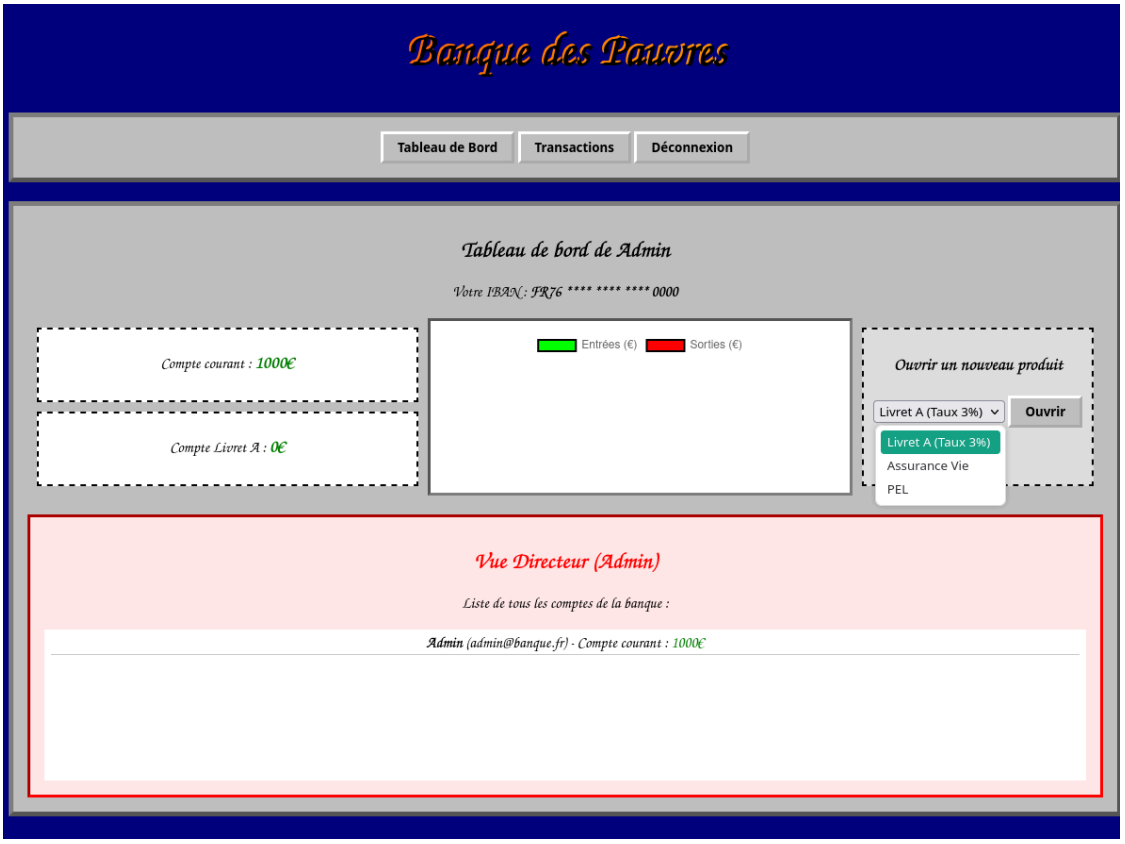
- **Pour l'inscription**

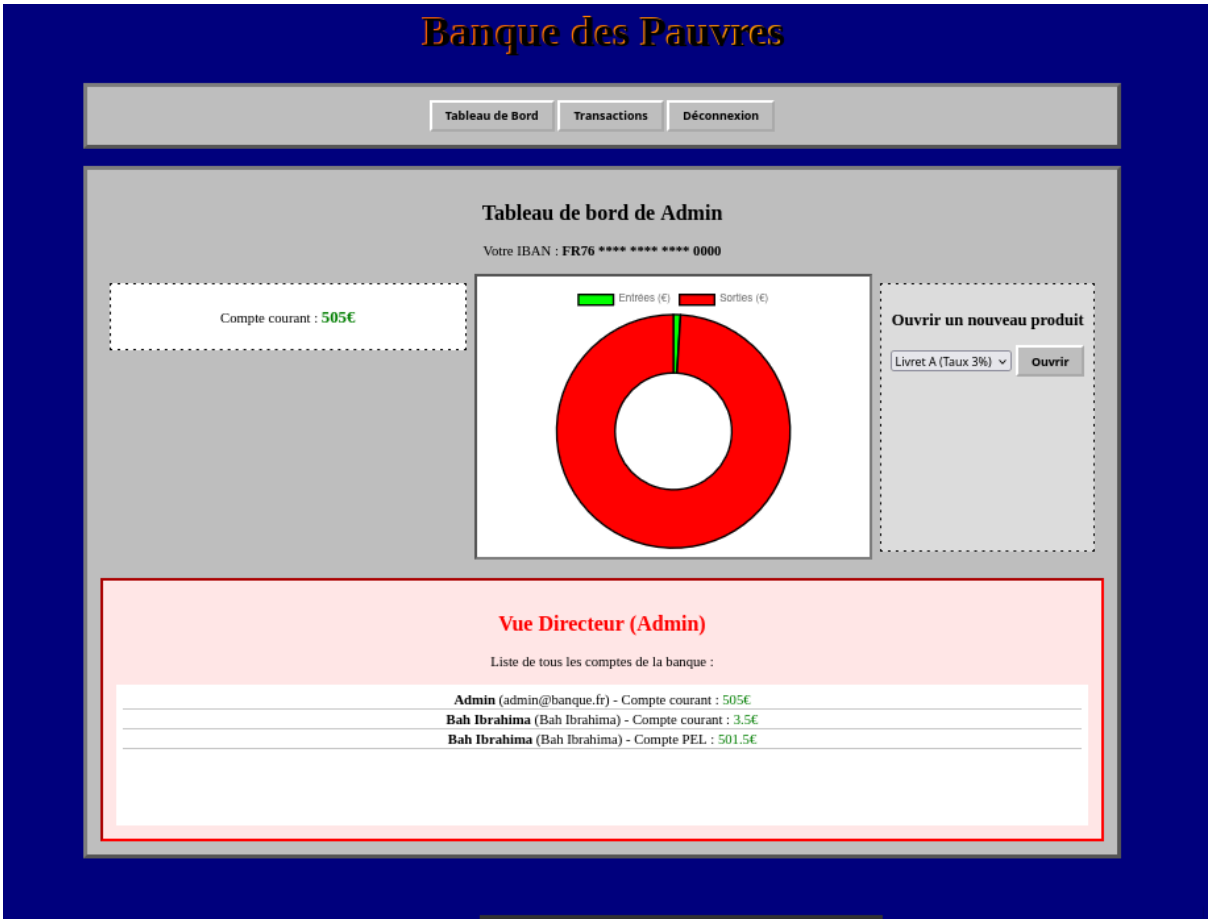
Cette page permet l'inscription. Pour cela il est nécessaire d'entrer un nom, un email, un âge strictement supérieur à 15 ans (notre banque s'adresse à un public de 16 ans minimum) et un mot de passe de minimum 8 caractères en une fois.

- **Pour la connexion**

Cette page permet la connexion. Pour cela, il est nécessaire d'entrer son email et son mot de passe. Le choix d'une adresse e-mail pour l'authentification est expliquée par le fait que celle-ci est en hypothétiquement unique.

B) Section du profil administrateur





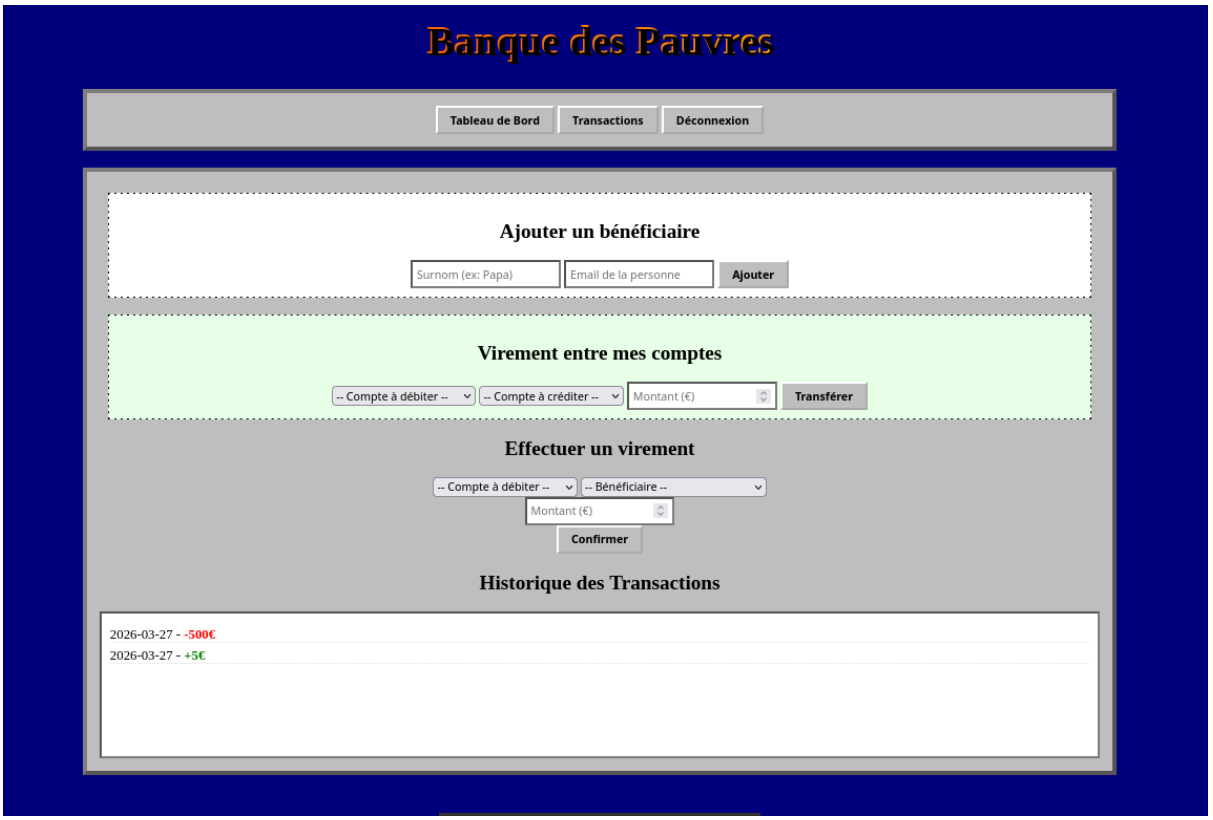
Le Compte Courant est initialisé avec le solde le plus élevé par défaut afin de faciliter les premiers tests sur l'application. Cela permet de simuler des opérations bancaires dès l'utilisation initiale de la plateforme.

L'utilisateur a également la possibilité de créer de nouveaux comptes, tels qu'un Livret A ou un compte EPL, afin de gérer différents types d'épargne au sein de l'application.

Le profil Administrateur dispose d'un accès étendu lui permettant de consulter l'ensemble des comptes des clients de la banque. Ce rôle peut être assimilé à celui d'un gestionnaire ou d'un technicien de l'application, chargé de superviser le bon fonctionnement du système.

Enfin, un graphique est prévu pour enregistrer de manière cyclique les entrées et sorties d'argent effectuées sur l'application bancaire. Ces données seront collectées à partir des opérations réalisées par les utilisateurs, notamment depuis le profil Administrateur présenté ici.

C) Section des transactions



La section des transactions propose trois options principales.

La première consiste à ajouter un bénéficiaire. L'utilisateur peut nommer son bénéficiaire comme il le souhaite, l'information la plus importante étant son adresse e-mail. En effet, celle-ci est considérée comme unique dans le système et permet d'identifier correctement la personne destinataire du virement.

La seconde option correspond au virement externe. Celui-ci permet d'envoyer de l'argent vers un bénéficiaire en dehors de la banque.

La troisième option est le virement interne à la banque. Dans ce cas, l'utilisateur sélectionne le compte source (src) depuis lequel le virement est effectué, puis indique le bénéficiaire auquel l'argent doit être transféré.

Enfin, l'utilisateur peut visualiser l'ensemble des transactions effectuées dans un historique. Cette fonctionnalité est rendue possible grâce à la persistance des données, notamment via l'utilisation de volumes.

II - La base de données

A) Schéma de BD et contraintes

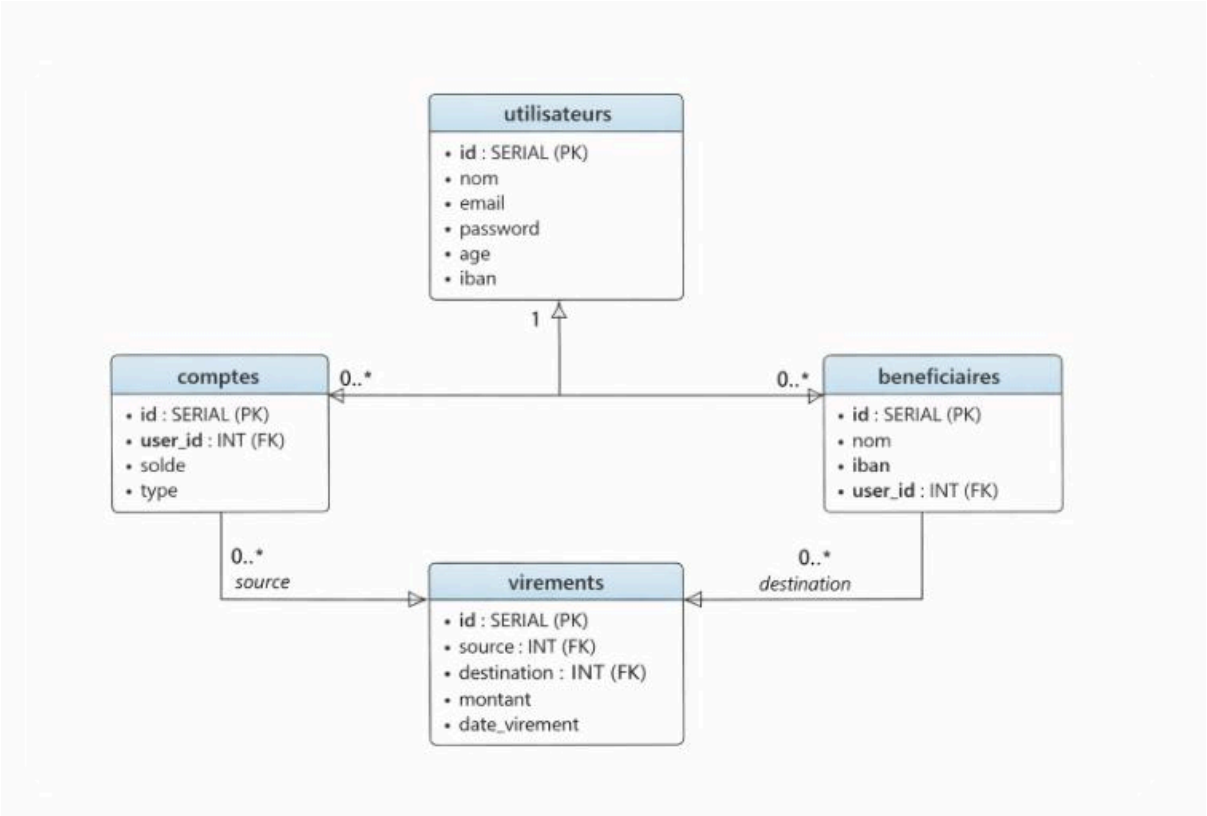
Notre base de données s'articule autour de 4 tables principales, reliées entre elles par des clés étrangères (*FOREIGN KEY*) pour garantir la cohérence des opérations :

- Table *utilisateurs* : Stocke les informations des clients (id, nom, email, password, age, iban). L'email et l'IBAN possèdent une contrainte UNIQUE pour éviter tout doublon dans le système.

- Table *comptes* : Gère les différents produits bancaires. Elle contient l'id, user_id (référence au client propriétaire), le type (courant, épargne, PEL) et le solde.

- Table *beneficiaires* : Permet à un utilisateur d'enregistrer ses contacts pour de futurs virements. Elle relie un user_id à un nom et un iban de destination.

- Table *virements* : Agit comme le livre de comptes de la banque. Elle enregistre chaque transaction avec une source (ID du compte à débiter), une destination (ID du compte à créditer), un montant et une date_virement.



Nous avons sécurisé les données directement dans PostgreSQL en ajoutant une contrainte *CHECK (solde >= 0)* pour empêcher les comptes de passer en négatif, et des clés étrangères *(REFERENCES)* pour garantir l'intégrité des virements.

B) Persistance des données (Volumes)

Un conteneur Docker est, par essence, temporaire : si celui-ci s'arrête ou redémarre, toutes les données enregistrées en son sein disparaissent. Imaginez l'impact pour une banque si les soldes des clients venaient à être perdus à chaque mise à jour - ce serait tout simplement inacceptable.

Pour éviter cette situation critique, nous avons mis en place un mécanisme de persistance grâce à un Named Volume (appelé db-data dans notre configuration). Ce volume permet de séparer le dossier de stockage de PostgreSQL (/var/lib/postgresql/data) du cycle de vie du conteneur, assurant ainsi sa sauvegarde physique sur la machine hôte.

De cette manière, même lorsque les conteneurs sont détruits, nos tables, utilisateurs et historiques de virements restent parfaitement conservés et sont automatiquement rechargés au prochain démarrage de l'application. Cette solution garantit la fiabilité et la continuité indispensable pour une gestion bancaire sécurisée.

III - Docker Python Backend

A) Choix technologique

Nous avons développé notre API (Interface de Programmation d'Application) en utilisant le framework Python **FastAPI**. Le fait de pouvoir gérer les requêtes de manière asynchrone est la raison pour laquelle on a préféré utiliser cette API.

Notre API agit comme un guichet virtuel qui traite les demandes du client (Frontend).
En effet, on va pouvoir contrôler à travers des règles qui créeront un compte en s'assurant que le client a bien plus de 16 ans et que son mot de passe fait au moins 8 caractères.

D'autre part, l'api attribue dynamiquement un bonus de bienvenue (10€ ou 15€ selon l'âge) et génère un IBAN français unique pour chaque nouvel utilisateur.

Puis, il est capable de sécuriser les transactions : la route */virement* vérifie que les comptes sources et destinataires sont valides et que les fonds sont suffisants avant d'interroger la base de données PostgreSQL via la bibliothèque *psycopg2* .

B) La Conteneurisation du Backend

Pour que notre code Python puisse s'exécuter n'importe où (sur le PC d'un développeur ou sur un serveur de production) sans erreur de dépendances, nous l'avons employé avec Docker.

- L'image Docker : Nous avons créé un Dockerfile spécifique pour le Backend. Cette image part d'une base Python officielle légère (alpine). Elle installe ensuite nos bibliothèques requises (comme fastapi, psycopg2, et le serveur web uvicorn) et copie notre code source (main.py) à l'intérieur.

- Le conteneur : Une fois l'image construite, Docker lance un conteneur. C'est un environnement totalement isolé de notre machine hôte. Le backend tourne à l'intérieur de cette bulle, écoutant les requêtes venant du réseau virtuel.

C) L'intégration des services annexes

L'architecture micro-services associée à Docker offre une grande modularité. Par exemple, un script indépendant nommé *interest.py* a été développé pour appliquer des intérêts sur les comptes d'épargne. Ce script fonctionne dans un conteneur distinct grâce à Docker Compose, s'exécutant de manière asynchrone sans ralentir ni bloquer l'API principale de la banque. Il accède à la même base de données via un réseau privé, illustrant ainsi la capacité de scalabilité du projet.

IV - Docker Appli Frontend

Le frontend permet à l'utilisateur de se connecter, consulter ses comptes, ajouter un bénéficiaire et effectuer des virements. Il communique avec le backend via des requêtes HTTP.

Organisation des fichiers :

- index.html : structure principale
- styles.css : design de l'interface
- main.js : logique et communication avec le backend
- nginx.conf : configuration du serveur Nginx
- Dockerfile : création de l'image Docker (image Nginx d'où la conf)

Notre Frontend est une Single Page Application (SPA) codée en HTML, CSS et JavaScript pur. Il est hébergé et distribué par un serveur web Nginx via un conteneur Docker dédié.

- **Dynamisme** : Au lieu de recharger la page à chaque clic, JavaScript interroge le Backend en arrière-plan (via l'API fetch) et met à jour l'interface en temps réel.
- **Visualisation** : Nous avons intégré la bibliothèque Chart.js pour générer un graphique interactif (Doughnut) qui s'actualise automatiquement en calculant les entrées, sorties et transferts internes à partir des données de l'historique.
- **Sécurité côté client** : Avant même d'envoyer des données au serveur, le JavaScript valide les saisies (ex: bordure rouge si le mot de passe est trop court ou si le virement est inférieur à 2€), ce qui optimise les performances réseau.

V - Docker compose

Le fichier docker-compose.yml permet de définir et d'orchestrer les différents conteneurs nécessaires au fonctionnement de l'application.

Un volume nommé db-data est utilisé afin d'assurer la persistance des données de la base de données. Cela signifie que les données sont conservées même si le conteneur de la base de données est arrêté ou recréé.

DB (PostgreSQL)

- Stocke toutes les données
- Variables d'environnement via .env pour sécurité et portabilité
- Volume db-data → persistance des données
- Réseau backend-net → isolé, non accessible depuis l'extérieur

Healthcheck (bonus)
Le conteneur PostgreSQL utilise un **healthcheck** avec pg_isready pour vérifier qu'il est prêt à accepter des connexions.

- Test toutes les 10 s : Docker vérifie l'état du conteneur toutes les 10 secondes
- Timeout 5 s : si la commande ne répond pas sous 5 secondes, le test échoue
- 5 retries : après 5 échecs consécutifs, le conteneur est considéré comme non fonctionnel

Les services backend et interest démarrent seulement si la base est **healthy** (depends_on: condition: service_healthy).

Cela garantit que les conteneurs dépendants ne tentent pas de se connecter avant que la base soit opérationnelle.

Backend (API)

- Communique avec la DB et fournit les données au frontend
- Dépend de la DB (depends_on + healthcheck)
- Connecté aux réseaux backend-net et frontend-net

Frontend

- Interface utilisateur exposée sur le port 80 (80:80)
- Dépend du backend
- Connecté au réseau frontend-net pour accéder au backend

Microservice Interest

- Service interne qui utilise la DB
- Réseau interne seulement (backend-net)
- Dépend de la DB (depends_on + healthcheck)

Réseaux

- frontend-net → communication frontend ↔ backend
- backend-net (internal) → DB + microservices internes, isolé pour sécurité
- Les deux réseaux sont de type bridge → chaque conteneur peut communiquer avec les autres du même réseau via leurs noms de service

Volumes

- db-data → persistance de PostgreSQL
- ./init.sql → script pour initialiser la base et reproduire les tables

Toute cette structure garantit une application résiliente (on ne perd pas de données), sécurisée (la DB est isolée) et scalable (on peut modifier le frontend sans toucher au reste).

VI - Sécurité

Cette dernière partie détaille la manière dont nous avons relié et sécurisé nos différentes applications (Frontend, Backend, Base de données) à l'aide de l'écosystème Docker. L'orchestration de ces conteneurs est gérée de manière centralisée par notre fichier docker-compose.yml.

1. Gestion des environnements et configuration (.env)
Pour des raisons de sécurité et de flexibilité, nous avons utilisé un fichier .env. Celui-ci a deux fonctions principales :

Centraliser les informations sensibles : Il permet de stocker les mots de passe (comme ceux de la base de données) et les variables de configuration sans les écrire en clair dans le code source.

Faciliter la maintenance : Il permet de modifier facilement les paramètres du docker-compose.yml sans avoir à éditer le fichier principal.

2. Architecture Réseau : Le principe du moindre privilège
Nous avons soigneusement mis en place un double réseau virtuel de type "Bridge" afin de respecter rigoureusement le principe de "moindre privilège" et de garantir une sécurité optimale des données bancaires :

Le réseau frontend-net (Zone Publique) : Ce réseau connecte directement le client (le navigateur web) au serveur Nginx. C'est la seule porte d'entrée accessible depuis l'extérieur sur notre machine hôte.

Le réseau backend-net (Zone Privée) : Configuré avec l'option internal: true, ce réseau relie exclusivement le Backend (l'API Python) et les scripts annexes à la Base de Données PostgreSQL.

Pourquoi cette architecture ?
Parce qu'elle rend la base de données totalement invisible depuis l'extérieur. Même en cas de tentative d'attaque sur l'adresse IP de la machine hôte, il serait impossible d'accéder au port 5432 de la base de données, puisque seul le conteneur Backend y a accès. Cette configuration assure ainsi une protection renforcée et fiable de l'application.

3. Asynchronisme et Micro-services (Le script interest.py)

L'un des grands avantages de Docker est la séparation des tâches. En plus de notre API principale, nous avons pu déployer un script indépendant (interest.py) dans son propre conteneur. Ce dernier se connecte au réseau backend-net pour appliquer automatiquement les intérêts sur les livrets d'épargne. Le fait de l'isoler garantit que ses calculs ne ralentiront jamais les requêtes des utilisateurs sur le Frontend.

4. Cycle de vie et Persistance des données
Nous pilotons l'ensemble de cette infrastructure grâce à quelques commandes clés :

sudo docker compose up --build -d : Permet de reconstruire les images (si le code a changé) et de lancer tous les conteneurs en arrière-plan.

sudo docker compose down -v : Arrête les conteneurs et supprime les volumes (-v). Nous avons appris que cette commande détruit la persistance des données (le volume db-data). Pour conserver les comptes bancaires et l'historique d'une session à l'autre, il faut exécuter cette commande sans l'option -v.

VII - Bonus

A) Docker secret

Pour sécuriser les informations sensibles de la base de données, nous avons utilisé **Docker Secret**. Les identifiants (user, password, database) sont stockés dans des fichiers .txt séparés et déclarés comme secrets dans le docker-compose.yml.

Les conteneurs PostgreSQL, backend et service d'intérêts accèdent à ces secrets via /run/secrets/. Le code Python lit ces fichiers pour se connecter à la base, avec un fallback sur les variables d'environnement en local. Cette méthode évite de mettre les mots de passe en dur et garantit que chaque service récupère uniquement ce dont il a besoin.

Étapes importantes :

- Création des secrets : mot de passe, nom de la base et user dans des fichiers .txt.
- Lecture dans Python : la fonction get_db_connection() utilise ces secrets pour se connecter à PostgreSQL.
- Déclaration et connexion dans docker-compose : les services déclarent les secrets qu'ils utilisent, et PostgreSQL lit directement ces fichiers au démarrage.

B) 4ème conteneur

Un quatrième conteneur a été ajouté afin de gérer les intérêts des comptes épargne.
Ce service s'exécute indépendamment du backend et applique périodiquement un taux d'intérêt de 3 % aux comptes de type *épargne* tous les 24h.

Il se connecte directement à la base de données PostgreSQL et met à jour les soldes des comptes concernés à l'aide de requêtes SQL exécutées automatiquement.

Cette séparation du service de calcul des intérêts permet de le rendre autonome, sans impacter le fonctionnement du backend ou des autres composants de l'application.

C) Implémentation healthcheck

cf **Docker compose**